BNN_FCC Accelerator Design Contest

Pawin Ruangkanit
RC2 Apple Design Contest
Gainesville, Florida, USA
ruangkanitpawin@ufl.edu

1 Summary

1.1 Results

Table 1 summarizes the explored design points. The first three columns specify the architectural parameters used in each run: input bus width, parallel inputs, and per-layer neuron partitioning. Several design points exceed the maximum operating frequency typically associated with BRAM on the target device. However, these results were achieved using LUTRAM rather than BRAM, indicating that the datapath itself is capable of higher clock frequencies when not limited by BRAM timing. Furthermore, all results were obtained from an out-of-context implementation, which excludes board-level I/O and other full-system constraints. Consequently, these frequencies should be interpreted as optimistic upper-bound estimates rather than fully realizable system-level results. Following the FINN-style reporting convention, FPS is computed from the steady-state throughput:

$$\text{FPS} = \frac{F_{\max}}{\text{cycles/output}}$$

where $F_{\max}$ is in Hz. The latency column is still reported separately as the average cycles per image. Among the explored points, the best measured implementation was the 256-bit / 128-input / 64, 64, 10 configuration, which reached 15.56M FPS. The FINN paper reports a maximum of 12.3 million image classifications per second on MNIST, so this best measured point is 1.27× higher, or about 26.5% above that published result. This comparison should still be interpreted carefully, since the values reported here come from an out-of-context LUTRAM-based implementation rather than a full system build.

Table 1: Current design-space results summary.

Input Bus Width	Parallel Inputs	Parallel Neurons	FMAX	Cycles/Output	Latency	FPS
64	32	64,32,10	722	104.8	290	6.89M
128	64	64,32,10	635	56.1	195.2	11.32M
256	128	64,64,10	501	32.2	130.2	15.56M
512	128	256,64,10	—	17.8	95.2	—
1024	128	256,64,10	—	10.6	78.4	—

Table 2: Resource utilization for the top three measured design points.

Input Bus Width	Parallel Inputs	Parallel Neurons	LUT	Registers	LUTRAM	BRAM
64	32	64,32,10	27723 (1.60%)	46757 (1.35%)	8625 (0.50%)	0 (0.00%)
128	64	64,32,10	42518 (2.46%)	60436 (1.75%)	8281 (0.48%)	0 (0.00%)
256	128	64,64,10	67663 (3.92%)	107002 (3.10%)	9037 (0.52%)	0 (0.00%)

Important Judging Notes. Use -p 1.25 in OpenFLEX when building the design. I found this to be most consistent as Vivado gives up early otherwise. Also, I was not able to get the openFLEX timing to work, as I am on a Windows device and the commands did not work, I tried to patch the tool to make it work on windows, but then Modelsim would hang forever. To verify results please use make sim-original-tb.

1.2 Implementation and Tradeoffs

The implementation target was to maximize throughput first and then optimize for clock frequency. Another design priority was parameterization. I wanted every component to be parameterized to be fully scalable to any power-of-two amount of parallel neurons, parallel inputs, and input width. This allowed me to gather these results for various configurations as I scaled up the input bus width.

Throughput is increased by replicating neuron processors and the corresponding weight and threshold RAM banks, allowing more

neurons to be processed in parallel each cycle. However, that same replication creates difficult timing paths because the RAM-side control and data signals become large fanout nets with significant routing cost. To address this, multiple pipeline stages were inserted on the interfaces that communicate with the weight and threshold RAM units, breaking these long paths into shorter stages and improving timing closure.

A similar approach is used in the compute reductions. The XNOR accumulation and argmax logic are implemented as recursive balanced trees, which makes it natural to pipeline the computation between tree levels. This reduces the logic depth of each stage and allows the design to scale to wider buses and larger parallel neuron counts without collapsing FMAX. The design is also heavily parameterized, which made it possible to evaluate the architecture across a range of bus widths, parallel-input counts, and neuron configurations while preserving the same overall structure.

The design also supports functionality beyond what was tested for the project. Notably, the design supports partial configuration and input buses as specified by TKEEP, it also supports complete reconfiguration of weights and thresholds in between image streams, as well as customizable hidden layers and topologies.

1.3 Verification/Testbench

The base testbench was extended into a UVM flow to verify functionality beyond the nominal contest path and to provide more complete coverage of the design. Some of this behavior includes: partial TKEEP configuration and input streams, interleaved and reordered configuration traffic, weights-only and thresholds-only reconfiguration, partial-layer updates, deeper hidden-layer topologies, extreme threshold/weight/pixel values, long inter-transaction gaps, output backpressure, and reset/reconfigure behavior.

- Functional coverage was collected with `make coverage-sweep`.
- A text coverage report was generated with `make coverage-sweep-report`.
- The achieved-coverage report is written to `coverage/bnn_fcc_coverage_sweep_test_coverage.txt`.
- Supports partial TKEEP configuration and input streams.
- Supports reconfiguring weights and thresholds after a few images.
- Supports interleaved configuration streams, including thresholds-first and out-of-order layer updates.
- Supports deeper hidden-layer stacks; see `make sim-ten-hidden`.
- Tested long delay gaps and extreme weight, threshold, and pixel values.
- Scales to higher neuron counts when the input bus width increases.

The coverage sweep achieved 93.18% total covergroup coverage, covering 392 of 445 bins across 26 covergroup types, with 0 assertion failures in the reported run.

Table 3: Coverage summary from the directed UVM sweep.

Area	Measured Groups	Status	Main Open Bins
Config stream	toggle 100%, interface 100%, handshake 100%, packet 75.00%, header 80.00%, order 86.66%	Strong	Short or single config packets, plus a few header and ordering corner bins.
Image input	toggle 100%, interface 100%, handshake 100%, spacing 100%, image 62.50%	Strong	Only generic short-packet image-shape bins remain; the contest image path is covered.
Output stream	toggle 100%, interface 100%, backpressure 100%, pattern 100%, output 95.00%	Strong	All 10 classes were observed; remaining misses are a few first-class transition bins.
Config data diversity	weight density 100%, threshold 86.66%	Strong	Main miss is the threshold unknown ratio bin.
Computational stimulus	output class 100%, class transition cross 70.00%	Strong	Every class was produced; misses are mostly “first output was class X” bins.
Reconfiguration	reconfig 83.33%	Good	Full, weights-only, thresholds-only, and partial reconfiguration were hit; the open bin is reconfiguring all layers at once.
Reset / recovery	reset 53.75%, post-reset 100%	Partial	Reset timing was mostly exercised during configuration, not idle, image, output, or TLAST boundaries.

2 Top-Level Organization

This report is organized in the same order that the design is used in practice. The top level accepts a configuration stream that loads weights and thresholds, an input stream that carries activation data, and an output path that returns the classification result. The main body then follows two phases of operation: the configuration phase and the compute phase.

At the top level, `rtl/bnn_fcc.sv` connects three major subsystems: `config_manager`, `data_in_manager`, and `bnn`. The first prepares and distributes model parameters, the second reshapes incoming input data into the binary words expected by the network, and the third contains the replicated layer datapaths that perform inference.

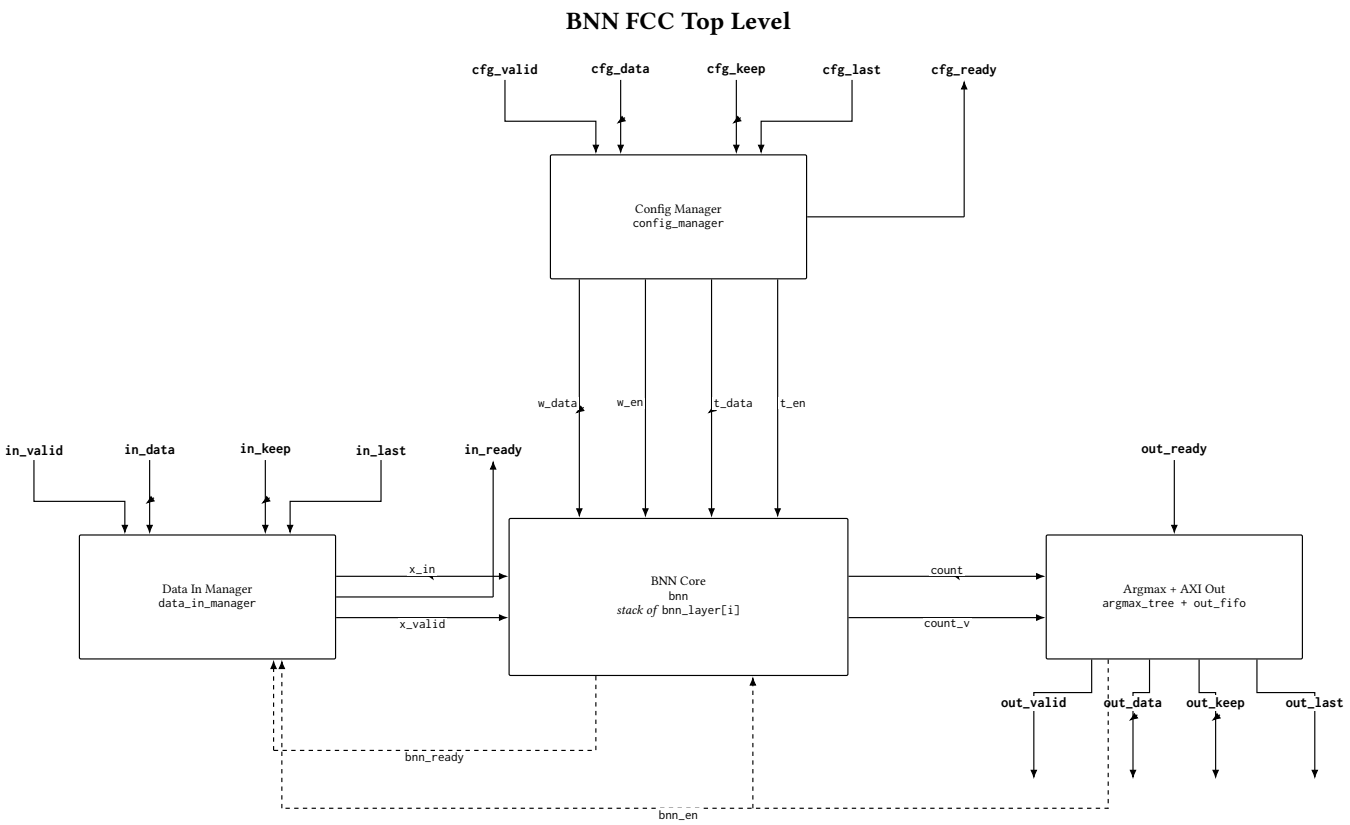

Figure 1: Top-level organization of `rtl/bnn_fcc.sv`. Configuration ingress, activation ingress, the BNN core, and the output side are shown with the main interface signals.

3 Configuration Phase

The configuration phase runs before inference and is responsible for converting the serialized parameter stream into aligned per-layer writes. In this design, that work is intentionally split into two levels. First, the top-level `config_manager` reshapes the incoming byte stream and separates weight payloads from threshold payloads. Then each `bnn_layer` uses a local `config_controller` path to distribute those writes across replicated RAM banks.

The top-level configuration datapath is implemented in `rtl/config_manager.sv` with support from `tkeep_byte_compactor`, `vw_buffer`, `config_manager_parser`, and `config_manager_pad_fsm`. Its job is to accept the external configuration stream, compact away invalid bytes, recover message structure, and emit write-oriented streams that are already separated into the weight and threshold paths.

The front end of this path is byte-oriented because the external interface can arrive with sparse valid-byte patterns. The parser then restores the higher level meaning of the stream by decoding the message header, choosing the weight or threshold destination, and generating the control metadata needed by the padding logic. On the weight side, the pad state machine inserts synthetic bytes so the downstream layer packers can emit aligned write words.

3.1 Parser Control

The parser in `rtl/config_manager_parser.sv` is responsible for recovering message structure from the serialized byte stream. It operates as a small three-state controller: `PARSE_HEADER` consumes the 16-byte header, `PARSE_PAYLOAD` streams the payload bytes, and `PARSE_DONE` waits for the rest of the datapath to drain the current message before the next header begins. In practice, this means the parser owns all protocol-oriented responsibilities: it extracts `msg_type` and `layer_id`, computes the expected payload length from the compile-time layer tables, and asserts `payload_start` exactly when the header has finished.

That separation is intentional. The parser does not try to solve alignment or packer timing at the same time it is decoding the stream. Instead, it emits a clean payload-byte stream plus a small set of control signals and lets the next block deal with write alignment. Splitting those responsibilities into `config_manager_parser` and `config_manager_pad_fsm` made the control easier to implement and easier to reason about, because one FSM is stream/protocol-facing while the other is write-format-facing.

3.2 Padding Control

The padding controller in `rtl/config_manager_pad_fsm.sv` handles the weight-side alignment problem after the parser has already identified the message. Its state machine is likewise explicit: `IDLE` latches the message metadata at `payload_start`, `READ` consumes real payload bytes, `PAD` emits synthetic `0xFF` bytes when a neuron payload does not end on a packer-word boundary, and `DRAIN` flushes the remaining staged bytes before the next message starts. During `PAD`, the FSM asserts `stall` back to the parser so that the protocol side does not advance while synthetic bytes are being inserted.

The `fifo_vr` primitive is what makes that padded-byte path practical. The parser first writes weight payload bytes into a byte-wide `fifo_weights_bytes`, then the pad FSM chooses whether the next byte forwarded downstream is a real FIFO byte or a synthetic padding byte. That single-byte stream is finally repacked by the layer-local `fifo_vr` packer FIFOs into the wider aligned write words expected by each layer. Using the same variable-read FIFO style for both staging and repacking keeps the byte injection mechanism simple: the pad FSM only has to control one downstream byte-write interface, even though the final writes leave as much wider words.

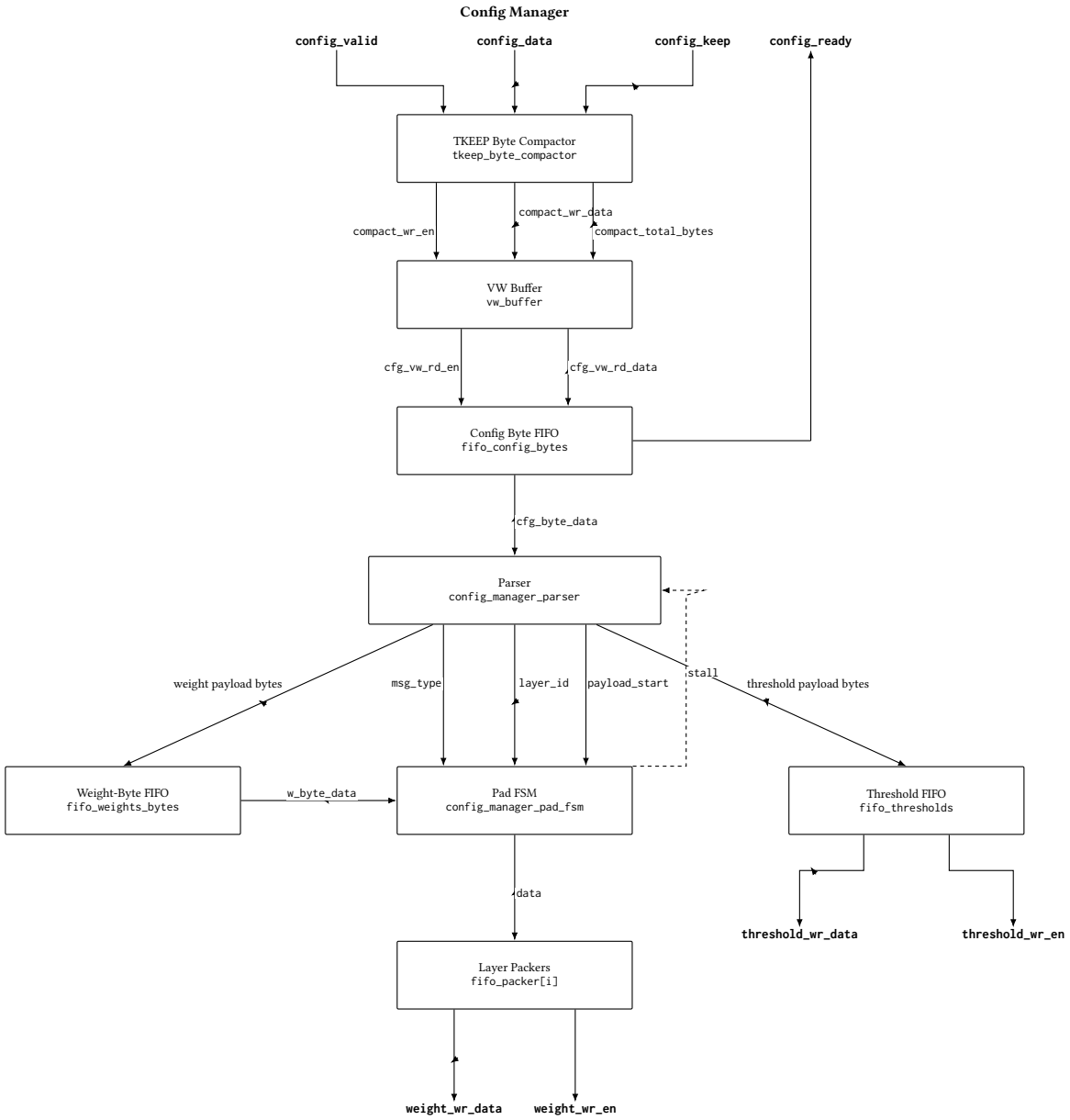

Figure 2: Block diagram of the `config_manager` datapath and its primary interface signals.

3.3 Layer-Local Write Distribution

After the top-level parser finishes that global reshaping step, each `bnn_layer` still has to fan the write stream out to replicated local memories. That is the purpose of `config_controller` and the surrounding staging registers in `rtl/bnn_layer.sv`. The controller generates the bank-select and address signals, while the aligned data stream bypasses the controller datapath and is retimed for fanout.

Those fanout pipeline registers are a central part of the implementation, not a minor detail. They cut the long controller-to-BRAM distribution path into a manageable stage while preserving a steady stream of writes after the pipeline fills. That is why this report treats the local write-distribution block as its own step in the overall configuration phase.

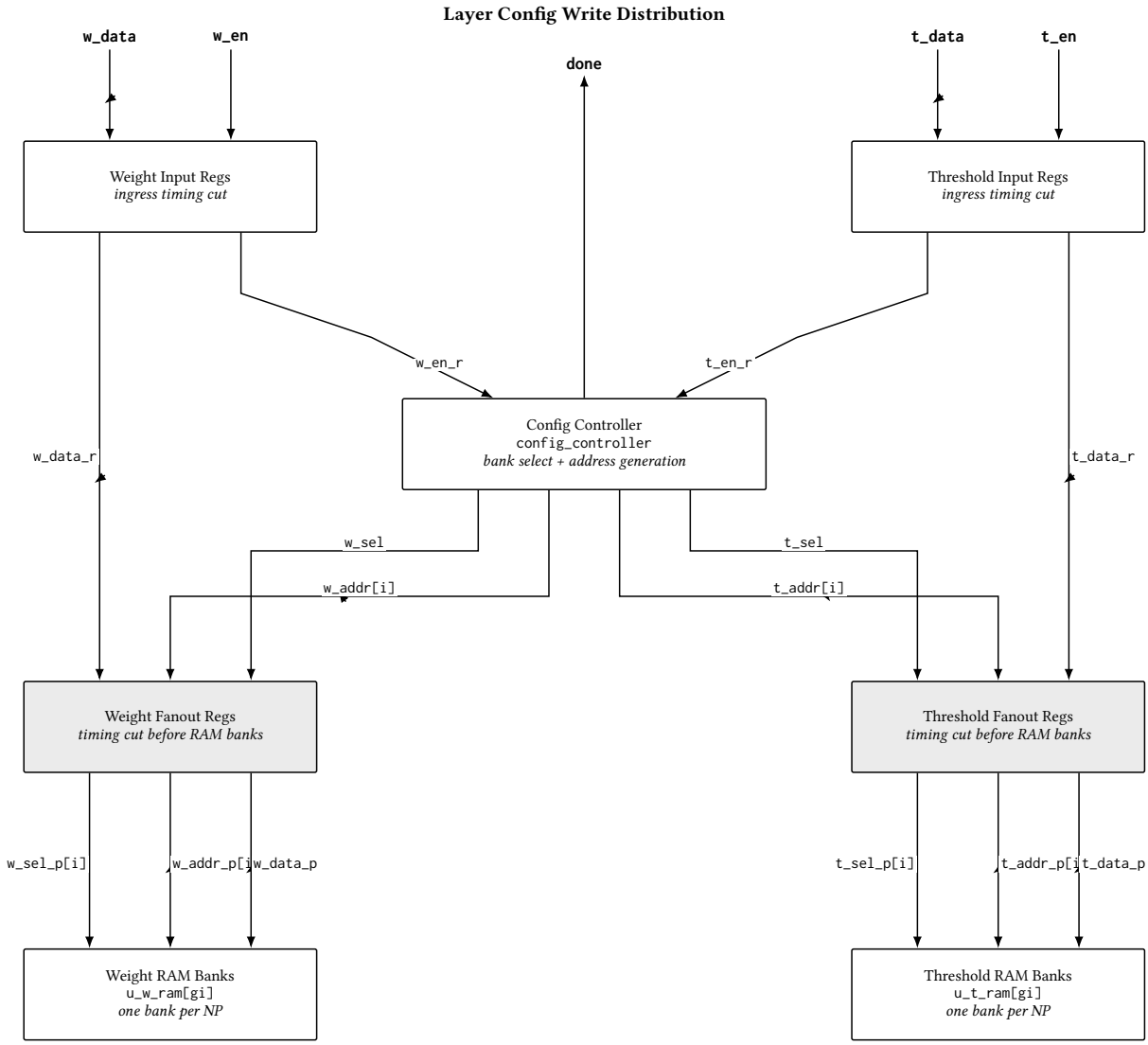

Figure 3: Layer-local config-write distribution around `config_controller`. The light gray fanout pipeline registers mark the timing-cut stage that drives the per-bank weight and threshold RAMs.

3.4 Timing Optimizations

This layer-local write-distribution block turned into one of the important timing problems in the overall design because it sits between a relatively compact control source and a wide set of physically distributed RAM banks. As the neuron count grows, the weight and threshold write enables, bank selects, addresses, and data all have to fan out across many destinations. That makes the path sensitive not only to logic depth, but also to placement and routing distance.

The solution used here was to instantiate the staging registers explicitly and preserve them with keep. Without that constraint, synthesis tended to absorb or merge the hand-placed pipeline stages back into neighboring logic, which removed the intended timing cut and recreated the same long fanout path. By keeping those registers structurally visible, the implementation preserves a clear controller stage followed by a separate fanout stage.

In practice, two register stages are used. The first stage captures the incoming write stream and controller-facing control signals near the local configuration ingress. The second stage sits at the RAM-distribution boundary and re-drives the per-bank write signals toward the replicated memories. That extra stage matters because the RAM banks can end up placed far apart after implementation; they are not guaranteed to remain close to the controller or to each other. With the additional retiming point, the placer and router have much more freedom to cluster the final fanout stage near the RAM banks, which is why this path no longer dominates the critical timing after optimization.

4 Compute Phase

Once the model parameters are loaded, the design switches to the compute phase. This phase again has two main pieces. The first is the input-preparation path in `data_in_manager`, which converts the incoming activation stream into the binary words expected by the network. The second is the layer-local forward datapath inside `bnn_layer`, where buffered activations, RAM reads, and neuron-processor outputs are coordinated.

4.1 Input Framing and Packing

`rtl/data_in_manager.sv` mirrors several of the same structural ideas used in the configuration path, but now for activation traffic rather than parameter traffic. The front end again begins with `tkeep_byte_compactor`, because the image stream can arrive with sparse valid-byte patterns. After compaction, local frame-count logic tracks how many real bytes have been accepted in the current image and determines whether the final beat needs zero padding so the frame length becomes a clean multiple of `PARALLEL_INPUTS`. That merged real-data/padding stream is then written through `vw_buffer`, which repacks the byte stream into full internal words.

This block is intentionally separate from the actual layer compute logic. By finishing byte compaction, frame shaping, and word packing at the top level, the `bnn` core sees a regular stream instead of having to carry AXI byte validity and end-of-frame corner cases deep into the runtime datapath.

4.2 Binarization and Staging

After repacking, `data_in_manager` converts each byte into one binary activation bit using the local binarization threshold and writes those bits into the internal `bin_fifo`. This FIFO is more than a convenience buffer: it decouples the image-side repacking path from the BNN-side `bnn_ready`/`bnn_en` handshake, and it provides the backpressure signal that throttles the upstream byte path before the staging queue overflows. In other words, the compute phase still starts with a small stream conditioning problem, and the FIFO is what isolates that problem from the first-layer datapath.

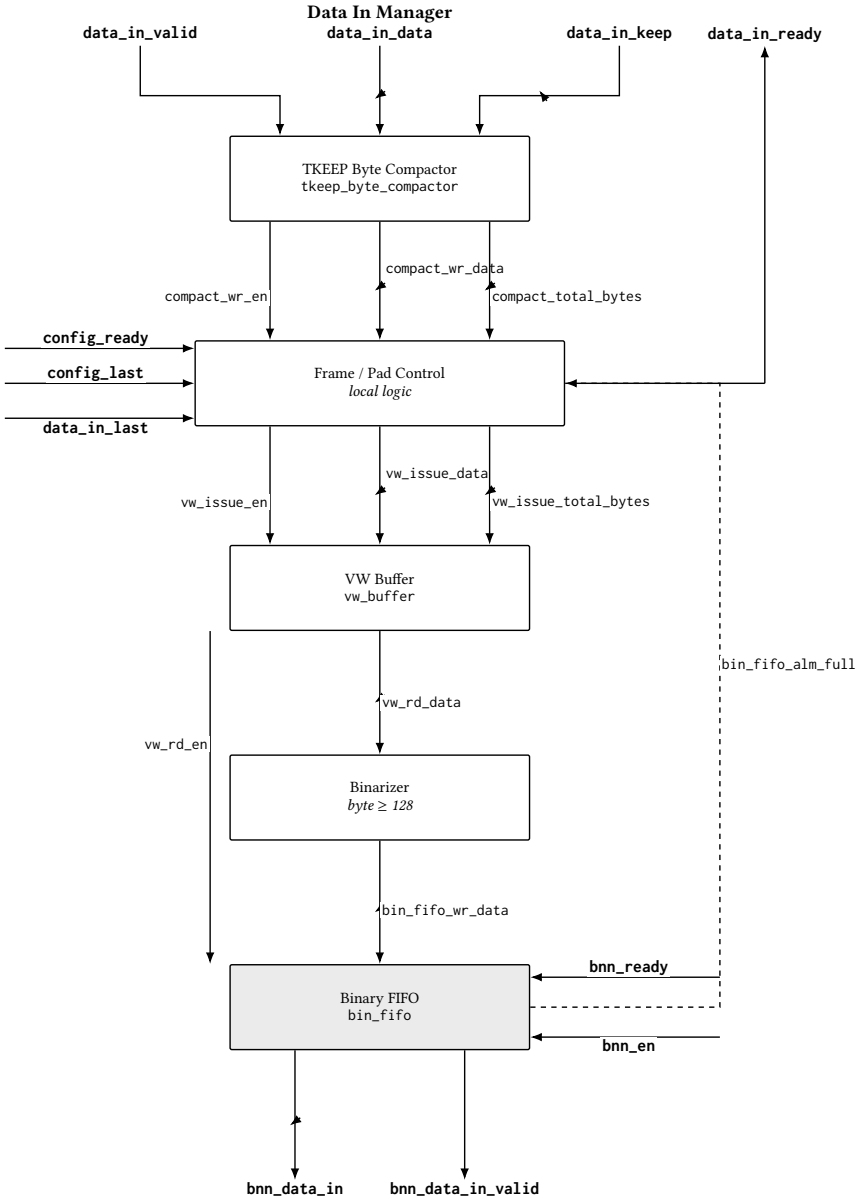

Figure 4: Block diagram of the `data_in_manager` datapath and its primary interface signals. The light gray staging block marks the BNN-facing queue that decouples input reshaping from the core compute flow.

4.3 Replay and Input Alignment

Inside `rtl/bnn_layer.sv`, the forward path begins with the local `replay_buffer`. This block allows one accepted activation frame to be reused across the repeated neuron-group computations required by the layer, instead of demanding the external source resend the same activations for every parallel-neuron slice. The replayed activation word then passes through the explicit alignment delay stage so it reaches the neuron processors in step with the parameter-read pipeline.

That alignment stage matters because the RAM path is not a single-cycle lookup. Between the controller scheduling, local per-bank address registers, and the registered RAM outputs, the weight and threshold operands emerge several cycles after the replay buffer launches the activation word. Keeping the input side explicitly delayed in the diagram makes that timing relationship much easier to understand than burying it inside the controller or RAM descriptions.

4.4 Read Control and Neuron Array

The second half of the compute phase is the parameter-read and evaluation path. `neuron_controller` sequences the shared weight and threshold read addresses, while the banked RAM structures provide one local operand stream per parallel neuron. The gray read-register stages shown in the diagram represent the staging/alignment points that keep the controller side, RAM side, and neuron-processor side synchronized. Finally, the `u_np[gi]` array consumes aligned activations, weights, and thresholds and produces the next layer's output bits along with the accumulator counts used by the last-stage argmax tree.

As in the configuration phase, the implementation is easier to follow because the responsibilities are divided cleanly: replay and alignment handle the activation side, the controller and RAM banks handle operand scheduling, and the neuron processors perform the actual XNOR-accumulate-and-threshold work.

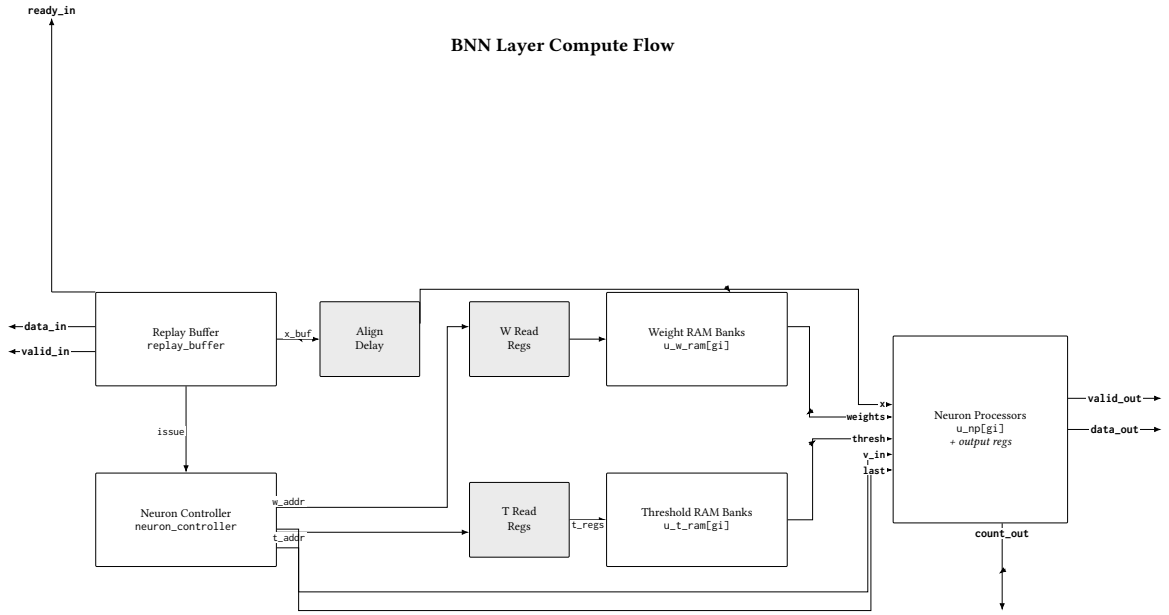

Figure 5: Simplified compute flow inside one `bnn_layer`. The light gray blocks mark the explicit staging and alignment registers in the activation and read-data paths.

4.5 Recursive Reduction Trees

Inside the neuron datapath, the XNOR-popcount reduction is implemented with the recursive pipelined tree in `rtl/xnor_add_tree.sv`, which then hands its partial counts to the recursive `rtl/add_tree.sv` reduction stage. The structure was inspired by Greg Stitt's recursive adder-tree approach on StittHub, but adapted here for the binary XNOR-accumulate workload used by the BNN layer. The main benefit of that recursive structure is that it naturally builds a balanced pipelined reduction instead of a long linear accumulation chain, so the reduction depth grows logarithmically with the parallel-input count.

That recursive implementation also creates explicit alignment delays so the shorter subtree outputs remain cycle-aligned with the deeper branches before they are added together. In practice, those generated delay stages had to be preserved with `keep`-style constraints, because synthesis would otherwise try to absorb them into neighboring logic and undo the intended pipeline structure. Keeping those delay registers intact was important for both functional alignment and timing, since the whole point of the recursive tree is to maintain a controlled staged reduction rather than let the tool collapse it back into a longer combinational path.

There is also one more explicit register cut after the tree output inside `rtl/neuron_processor.sv`. When the last partial sum for a neuron arrives, the final accumulated count and aligned threshold

are first captured locally, and only in the following cycle does the design perform the threshold-compare operation. That extra register stage splits the reduction path from the comparison logic so the tree output plus compare no longer form one combined critical path.

The output block uses the same general idea. The class-score reduction in `rtl/argmax_tree.sv` is also implemented as a recursive pipelined tree, with generated alignment delays on the shorter branch and a registered compare at each merge point. Using the same recursive reduction style for both the neuron popcount path and the final argmax path keeps the timing strategy consistent across the compute phase: balanced trees for depth control, explicit delay alignment between uneven subtrees, and preserved pipeline stages where the synthesis tool would otherwise optimize away the intended timing cuts.

5 Verification

The UVM environment was built by extending the existing directed `bnn_fcc_tb` flow rather than replacing it. The original testbench already had the important design-specific assets: the `BNN_FCC_Model`, configuration encoding, image stimulus generation, probability-based valid throttling, randomized output backpressure, and end-to-end pass/fail checking. The UVM work reorganized those capabilities into reusable layers so new tests could vary configuration order, partial reconfiguration, `TKEEP` packetization, reset timing, and image selection without rewriting low-level AXI signal code.

5.1 Top-Level UVM Wrapper

The top wrapper in `bnn_fcc_uvm_tb.sv` preserves the same runtime controls as the original testbench: topology selection, image count, model-verification enable, valid probabilities, backpressure mode, timeout, and clock period. It instantiates the DUT together with the three AXI4-Stream interfaces for configuration input, image input, and output, then publishes those interfaces through `uvm_config_db`.

One useful addition in the UVM flow is `bnn_fcc_ctrl_if`. That shared control interface gives tests, the scoreboard, and the coverage components a clean place to coordinate reset and output-ready forcing without directly reaching into the top module internals. The wrapper also keeps the output-side protocol assertions local to the signal layer, while the scoreboard handles the end-to-end functional checks.

5.2 Environment and Agents

The main environment in `bnn_fcc_env.sv` assembles three protocol agents, one scoreboard, and four coverage collectors. The configuration and image agents are active AXI4-Stream masters, while the output agent is passive because the DUT owns the output channel. This mirrors the real interface roles and keeps the output monitoring separate from output ready generation.

The environment forces all three monitors into packet-level mode before connecting them. That decision matters because the scoreboard and coverage components reason about full configuration messages, full image packets, and full outputs rather than isolated beats. The environment then connects each monitor to both the scoreboard and the configuration, input, output, and cross-stream system coverage blocks.

5.3 Sequence Architecture

Most of the reuse in the UVM flow lives in the shared sequence library `bnn_fcc_sequences.svh`. The configuration side is built around one common base sequence that reuses the shared `BNN_FCC_Model` to generate exactly the same legal configuration messages as the original testbench. Its key knobs select whether weights and thresholds are included, which layers are touched, and what message ordering is used. Together those controls cover the important reconfiguration scenarios: full configuration, partial configuration, weights-only updates, thresholds-only updates, and reordered traffic such as thresholds-first or layer-interleaved streams.

The image side follows the same pattern. A shared base image sequence loads stimulus from the same reference path used by the directed testbench, or generates randomized images for custom topologies, then packs those images into AXI beats. Thin derived classes change only the presentation style: beat-level traffic, packet-level traffic, scripted image-index traffic, scripted raw-pixel traffic, and `TKEEP`-repacked packet traffic. This is what keeps derived tests short: they choose a scenario, but they do not reimplement model knowledge or packet formatting.

5.4 Scoreboard and Reset Handling

The scoreboard in `bnn_fcc_scoreboard.svh` is state-aware because this design supports both reconfiguration and reset during test execution. Instead of comparing outputs immediately at the moment they appear, the scoreboard queues expected predictions when input packets arrive. Each queued entry stores the expected class, the logical image index, and a configuration epoch. Tests call `commit_model()` after sending a new configuration, and the scoreboard copies that model into an internal snapshot. That protects the checks from later test-side edits to the shared model handle.

Reset handling is also explicit. When reset asserts through `ctrl_if.rst`, the scoreboard clears all pending expectations, marks itself unconfigured, and ignores later outputs until a fresh post-reset model is committed. This avoids stale pre-reset expectations being compared against post-reset outputs. The same scoreboard also tracks average latency and average throughput using handshake timestamps reconstructed by the packet monitors.

5.5 Coverage-Oriented Tests

Because the infrastructure is layered, focused tests stay short. A test only needs to choose a configuration sequence flavor, an image sequence flavor, timing stress knobs, and optional reset or reconfiguration events. The shared environment handles the agent connections, the scoreboard model snapshots, and the coverage collection. The dedicated `bnn_fcc_coverage_sweep_test` then serializes many of these directed scenarios into one judge-friendly run, producing one simulation log, one UCDB, and one text coverage report.

The result is a verification flow that is specific to `bnn_fcc` rather than a generic UVM demo. The original model and stimulus logic are still the source of truth, but UVM turns them into reusable protocol-layer components, scenario-driven sequences, and a reset-aware scoreboard that can support a much broader set of verification scenarios than the original monolithic testbench.